INDEX IN DATABASE

- A database index is a DATA STRUCTURE that improves the SPEED OF DATA RETRIEVAL operations on a database table.

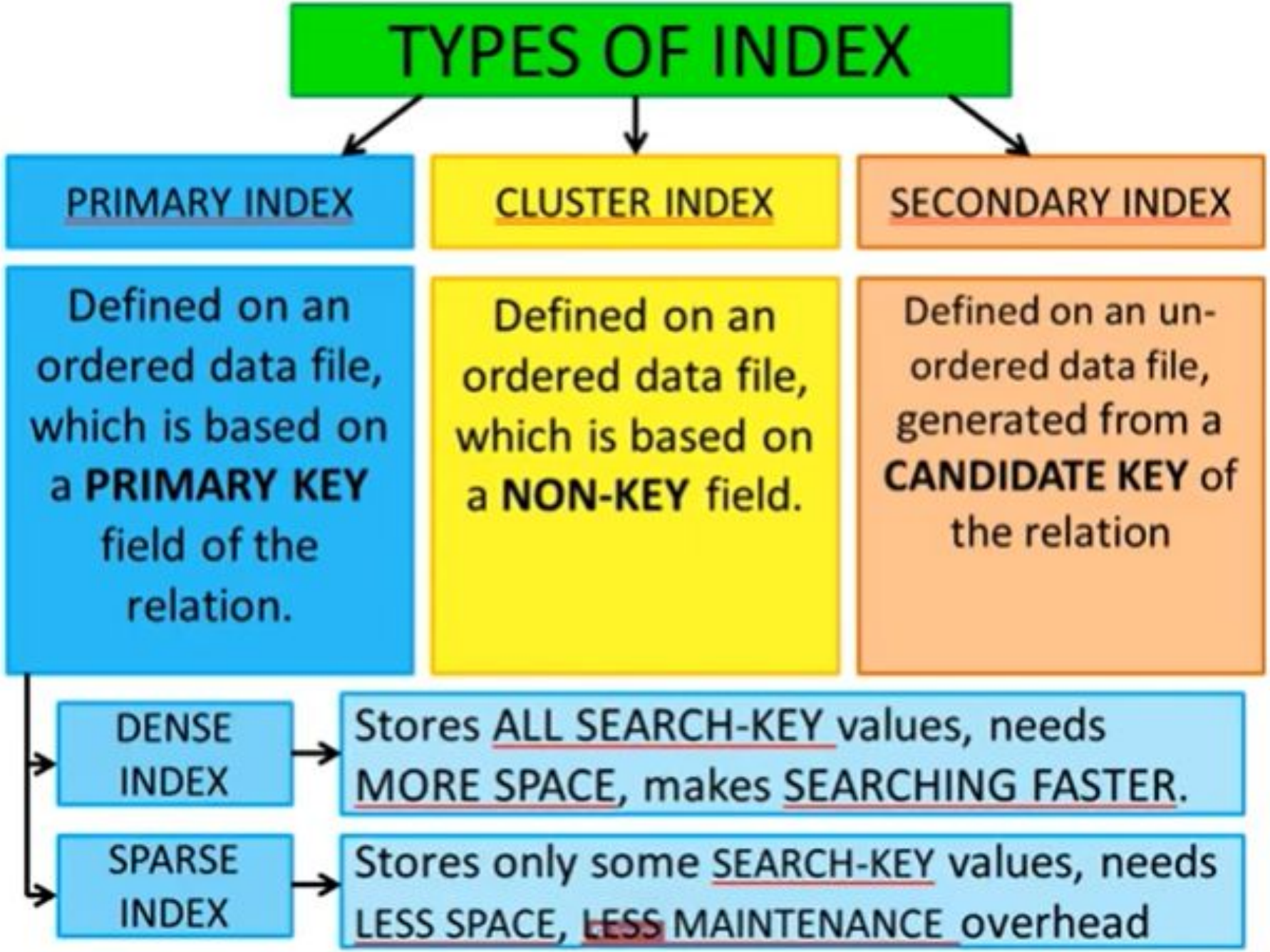
- It is used to LOCATE AND ACCESS the data in a database table QUICKLY, this process is called as INDEXING.

- Indexing is used to OPTIMIZE the performance of a database by MINIMIZING THE NUMBER OF DISK ACCESSES REQUIRED when a QUERY IS PROCESSED.

INDEX IN DATABASE

- Indexes are created using a FEW DATABASE COLUMNS.
- An INDEX is a small table having ONLY TWO COLUMNS.
- The FIRST COLUMN comprises a copy of the PRIMARY OR CANDIDATE KEY of a table. Its SECOND COLUMN contains a SET OF POINTERS for holding the ADDRESS of the DISK BLOCK where that SPECIFIC KEY value stored.




TYPES OF INDEX
PRIMARY INDEX
CLUSTER INDEX
SECONDARY INDEX
Defined on an ordered data file, which is based on a PRIMARY KEY field of the relation.
Defined on an ordered data file, which is based on a NON-KEY field.
Defined on an un-ordered data file, generated from a CANDIDATE KEY of the relation
DENSE INDEX
Stores ALL SEARCH-KEY values, needs MORE SPACE, makes SEARCHING FASTER.
SPARSE INDEX
Stores only some SEARCH-KEY values, needs LESS SPACE, LESS MAINTENANCE overhead

INDEX IN DATABASE

Indexing is defined based on indexing attributes, they are:

1. **ACCESS TYPES:** This refers to the type of access such as value based search, range access, etc.
2. **ACCESS TIME:** It refers to the time needed to find particular data element or set of elements.
3. **INSERTION TIME:** It refers to the time taken to find the appropriate space and insert a new data.
4. **DELETION TIME:** Time taken to find an item and delete it as well as update the index structure.
5. **SPACE OVERHEAD:** It refers to the additional space required by the index.

1. Primary Index

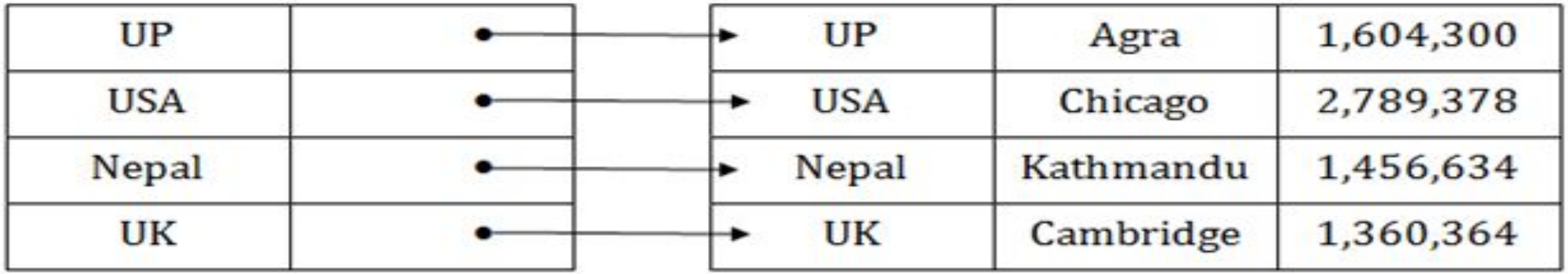
- If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.
- As primary keys are stored in sorted order, the performance of the searching operation is quite efficient.
- The primary index can be classified into two types

  (i) Dense index

  (ii) Sparse index

Dense index

- The dense index contains an index record for every search key value in the data file. It makes searching faster
- In this, the number of records in the index table is same as the number of records in the main table
- It needs more space to store index record itself. The index records have the search key and a pointer to the actual record on the disk



INDEX – DENSE INDEX

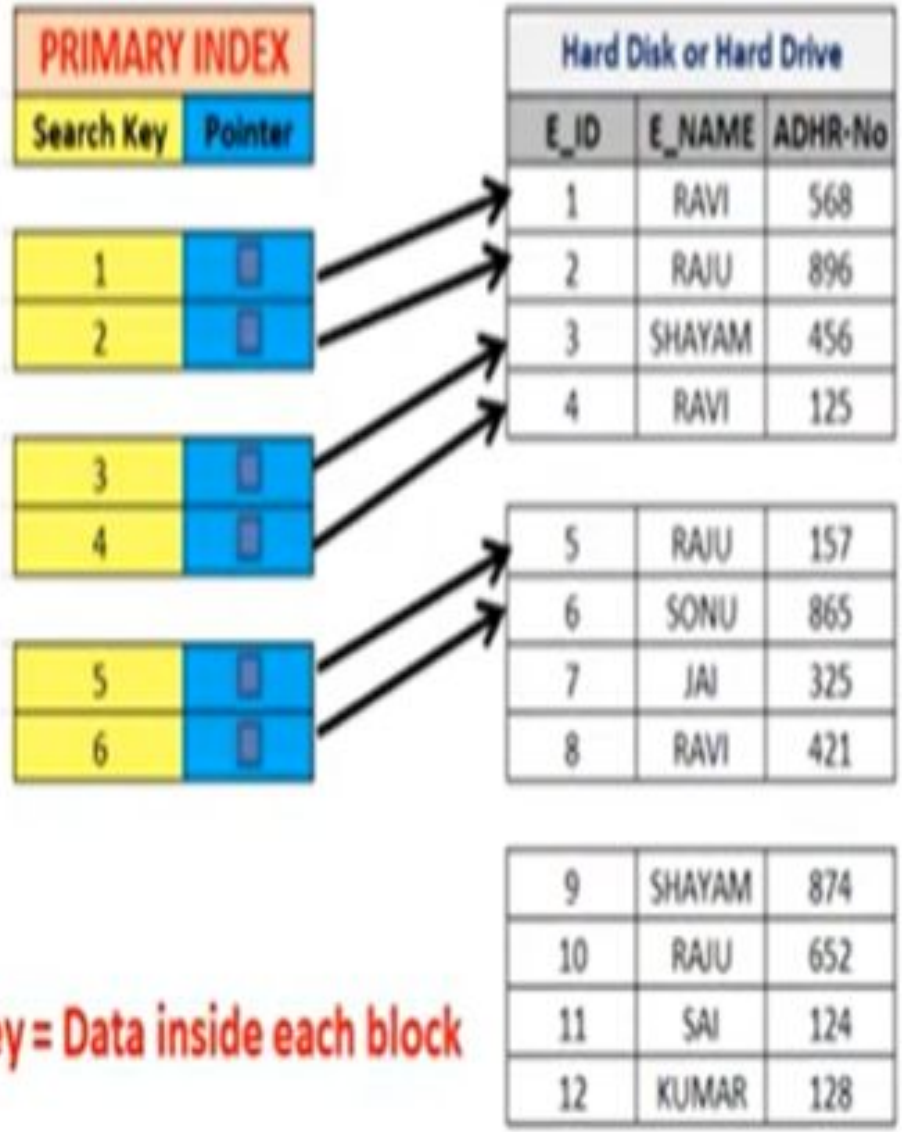
- Data is SORTED based on Primary Key.
- Dense Index Stores ALL SEARCH-KEY VALUES.
- Needs MORE SPACE and MAKES SEARCHING FASTER.
- Index records contain SEARCH KEY VALUE and a POINTER to the ACTUAL RECORD on the DISK.

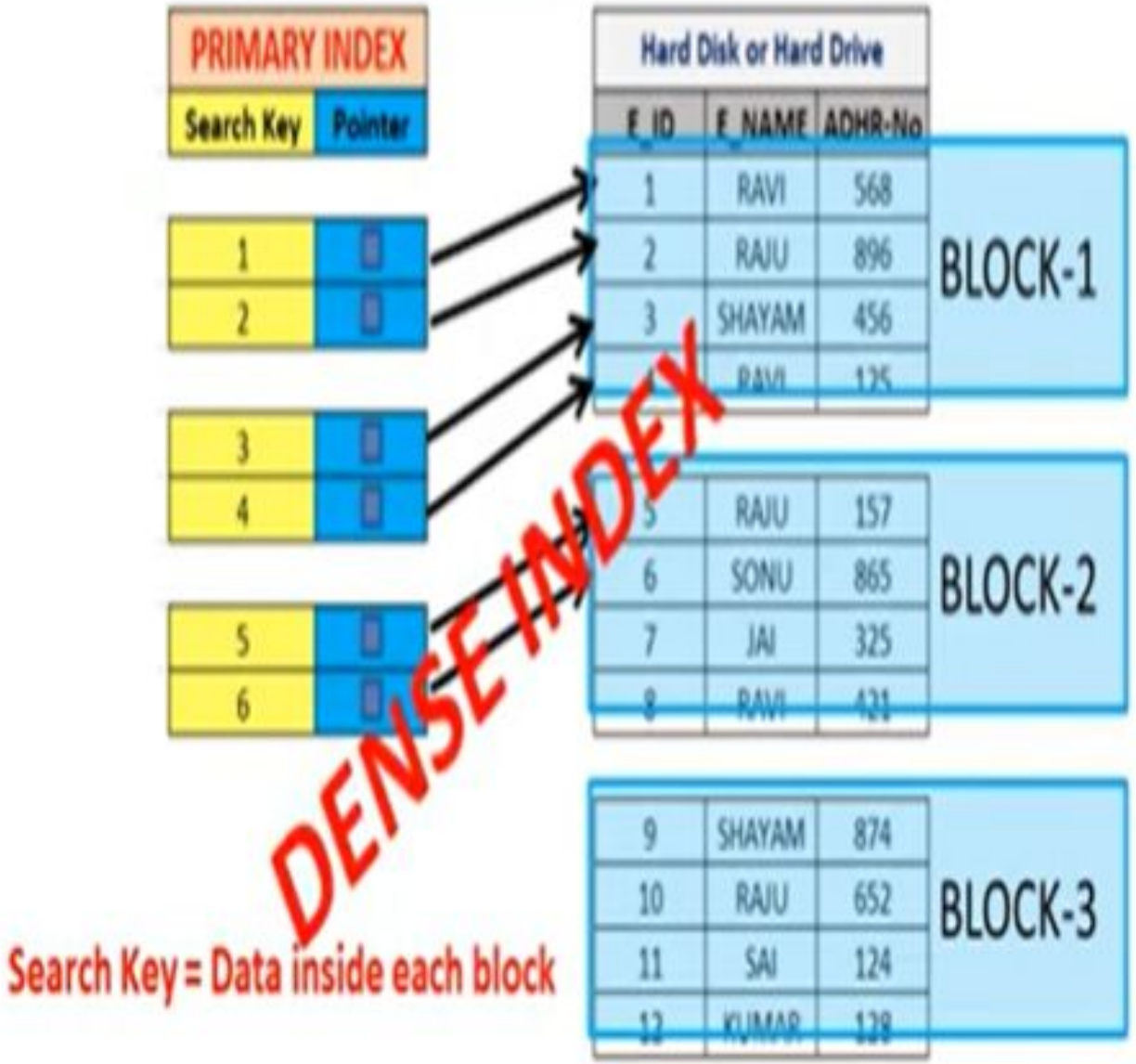


Hard Disk or Hard Drive

E_ID	E_NAME	ADHR-No
1	RAVI	568
2	RAJU	896
3	SHAYAM	456
4	RAVI	125
5	RAJU	157
6	SONU	865
7	JAI	325
8	RAVI	421
9	SHAYAM	874
10	RAJU	652
11	SAI	124
12	KUMAR	128

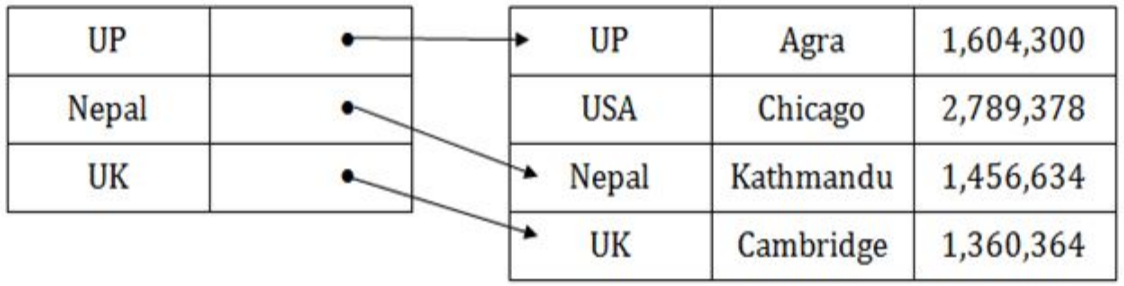
INDEX – DENSE INDEX



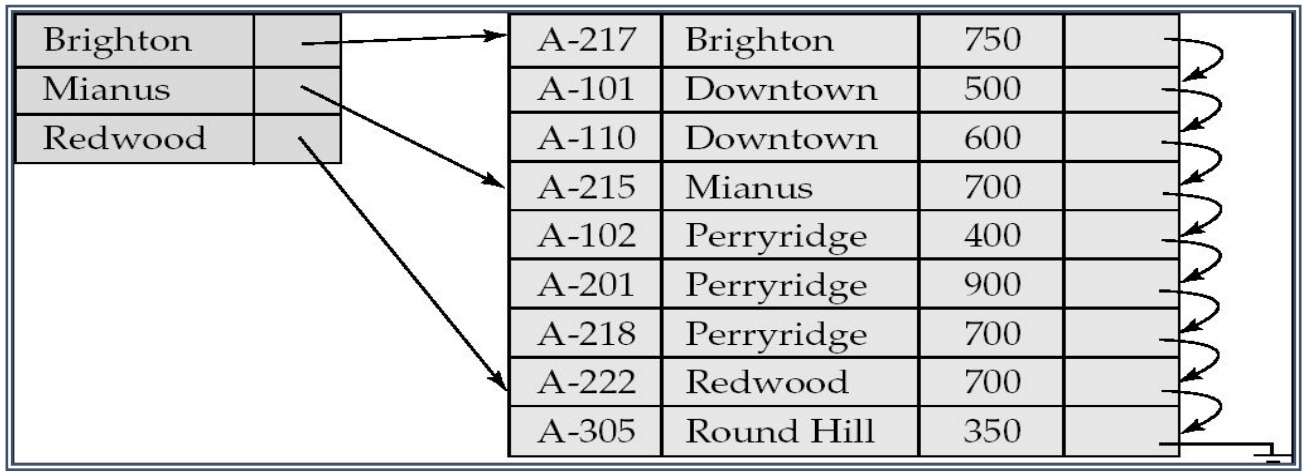


Sparse index

- In the data file, index record appears only for a few items. Each item points to a block.
- In this, instead of pointing to each record in the main table, the index points to the records in the main table in a gap



- To locate a record with search-key value *K* we:
- Find index record with the **highest entry which is smaller than or equal to k.**
- Search file sequentially starting at the record to which the index record points



PRIMARY INDEX – SPARSE INDEX

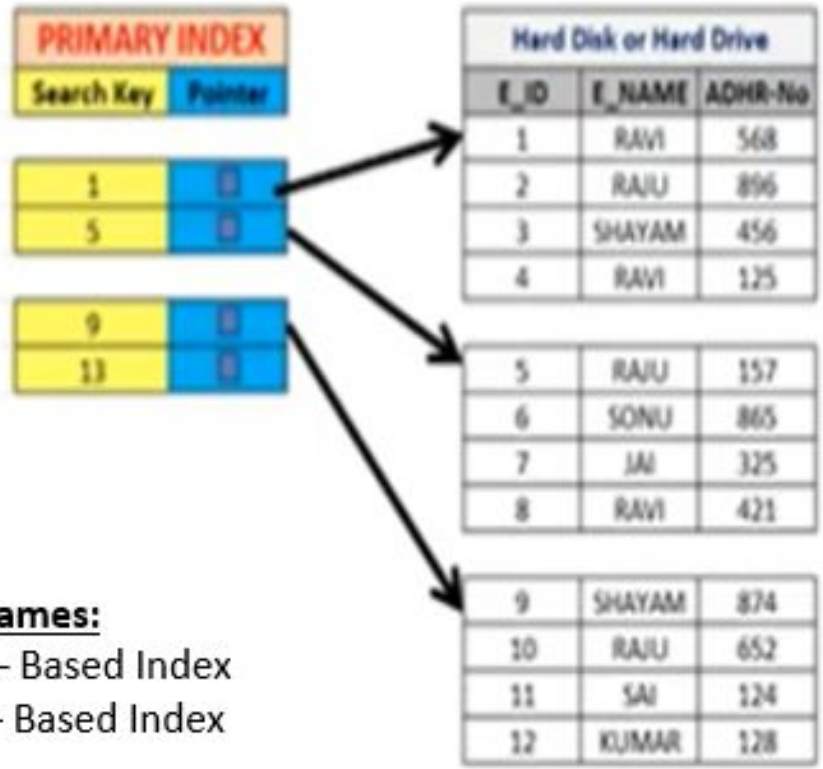
- Data is SORTED based on Primary Key
- Stores ONLY SOME SEARCH-KEY values.
- Needs LESS SPACE, LESS MAINTENANCE overhead.
- In Sparse indexing technique, a RANGE OF INDEX COLUMNS stores the same DATA BLOCK ADDRESS, and when data needs to be RETRIEVED, the block ADDRESS will be FETCHED.



Hard Disk or Hard Drive

E_ID	E_NAME	ADHR-No
1	RAVI	568
2	RAJU	896
3	SHAYAM	456
4	RAVI	125
5	RAJU	157
6	SONU	865
7	JAI	325
8	RAVI	421
9	SHAYAM	874
10	RAJU	652
11	SAI	124
12	KUMAR	128

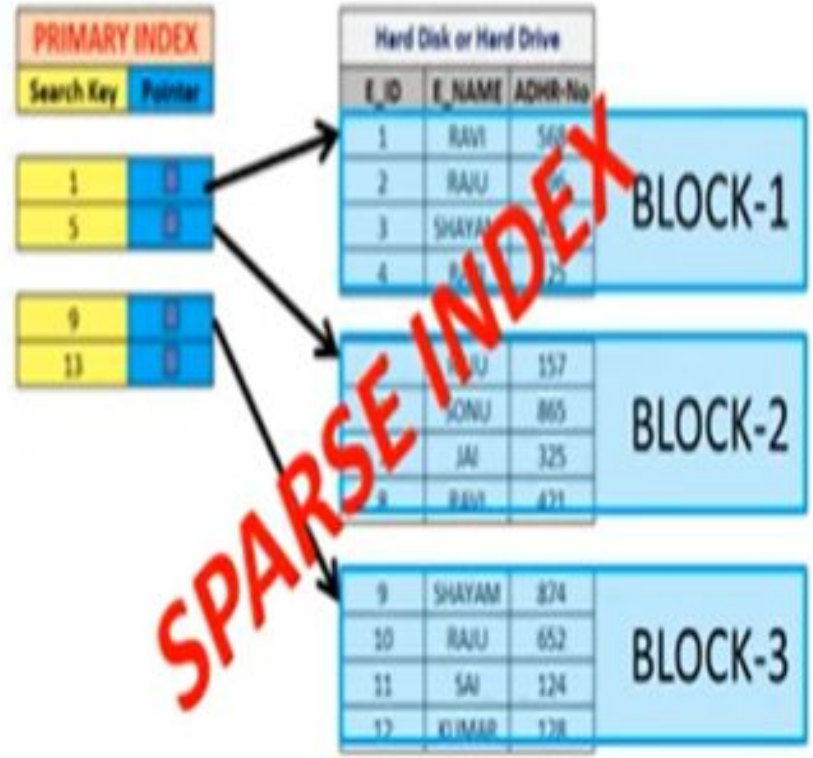
PRIMARY INDEX – SPARSE INDEX



Other Names:

Anchor – Based Index

Leader – Based Index



Search Key = Number of Blocks

2. Clustered Index

- A clustered index can be defined as an ordered data file. Sometimes the index is created on non-primary key columns which may not be unique for each record.
- In this case, to identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called a clustering index.
- The records which have similar characteristics are grouped, and indexes are created for these group.

INDEX – CLUSTERED INDEX

When to use Clustered Index ?

1. Ordered Data
2. Data already have Primary Index
3. Non-Key based search required.

- Mostly clustered Index will have COMBINATION OF NON KEY ATTRIBUTE ALONG WITH CANDIDATE KEY of the tables.
- Clustered Index is a SPARSE INDEX TYPE, where only ONE SEARCH KEY VALUE per block.

INDEX – CLUSTERED INDEX

Dept No	E_Name	Pointer
1	A	
1	B	
2	A	
3	B	
3	D	
3	A	
4	C	
4	B	
4	D	
4	E	
5	G	
5	A	
5	B	
5	F	
5	R	

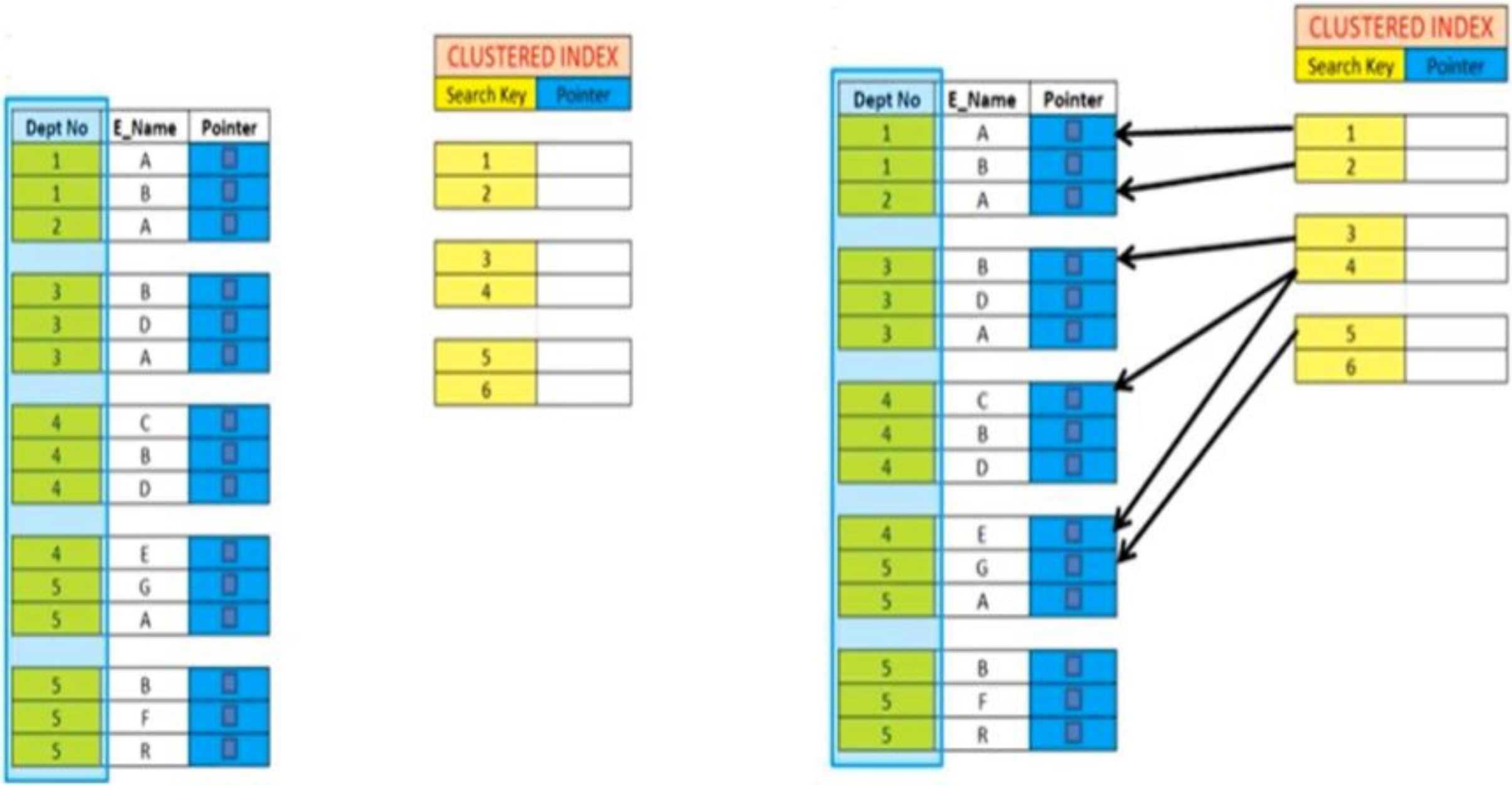
CLUSTERED INDEX	
Search Key	Pointer
1	
2	
3	
4	
5	
6	

Duplicate Data

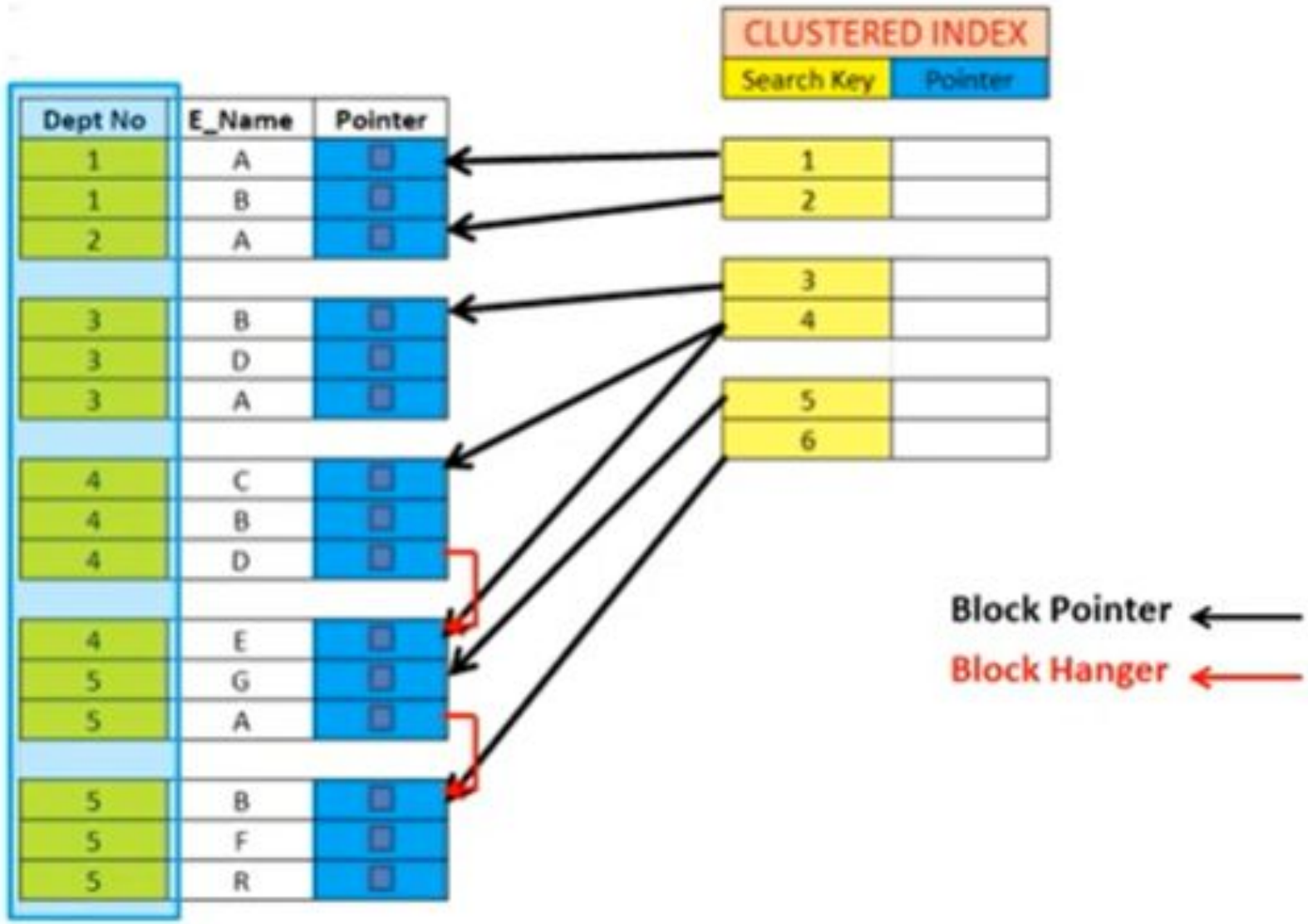
Dept No	E_Name	Pointer
1	A	
1	B	
2	A	
3	B	
3	D	
3	A	
4	C	
4	B	
4	D	
4	E	
5	G	
5	A	
5	B	
5	F	
5	R	

CLUSTERED INDEX	
Search Key	Pointer
1	
2	
3	
4	
5	
6	

INDEX – CLUSTERED INDEX

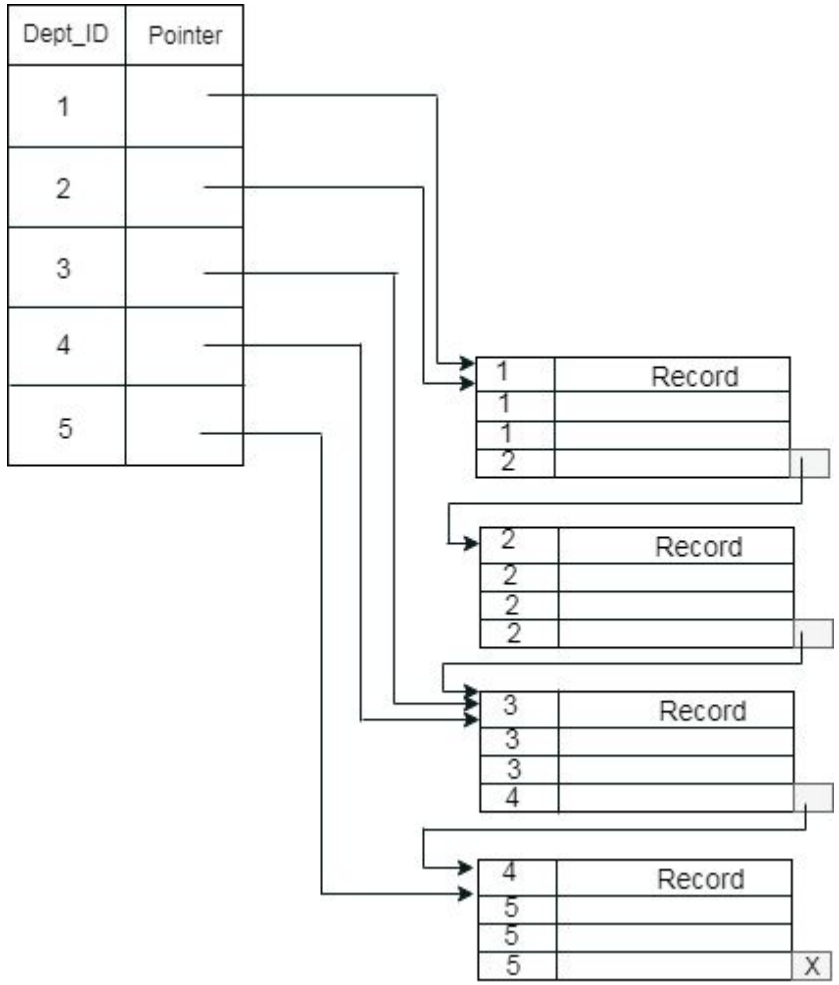


INDEX – CLUSTERED INDEX

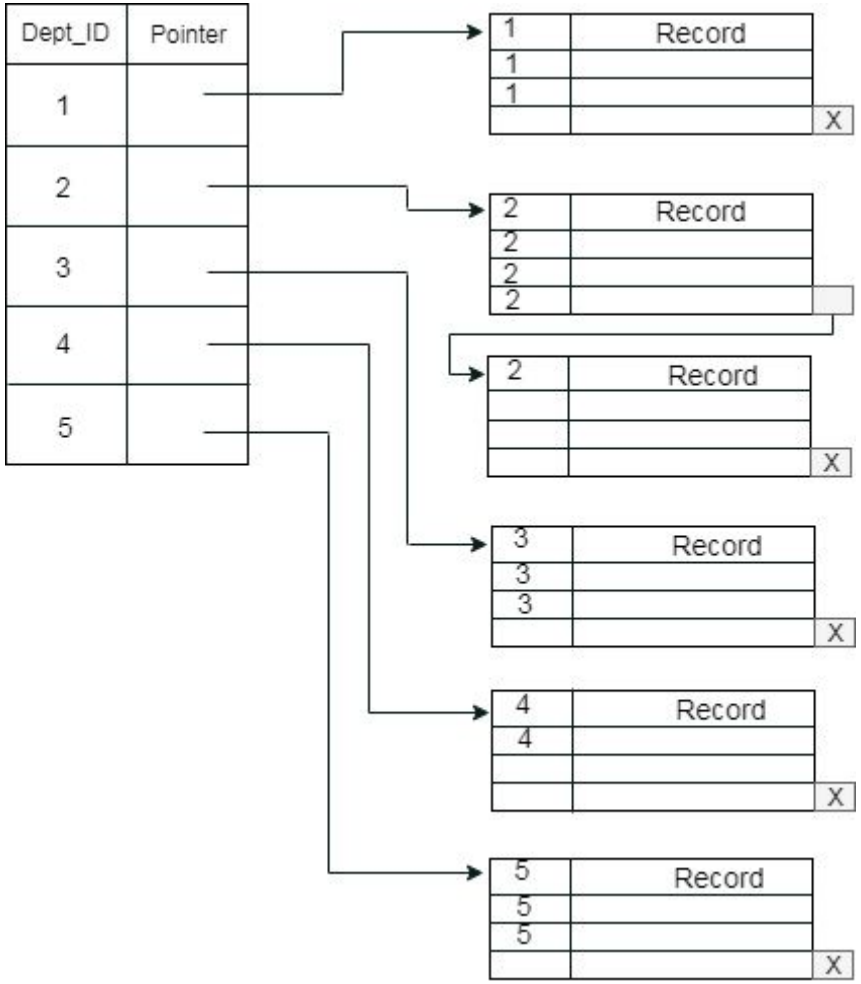


2. Clustered Index

Example: suppose a company contains several employees in each department. Suppose we use a clustering index, where all employees who belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole. Here Dept_Id is a non-unique key.



The previous schema is little confusing because one disk block is shared by records which belong to the different cluster. If we use separate disk block for separate clusters, then it is called better technique.



INDEX – SECONDARY INDEX

When to use Secondary Index ?

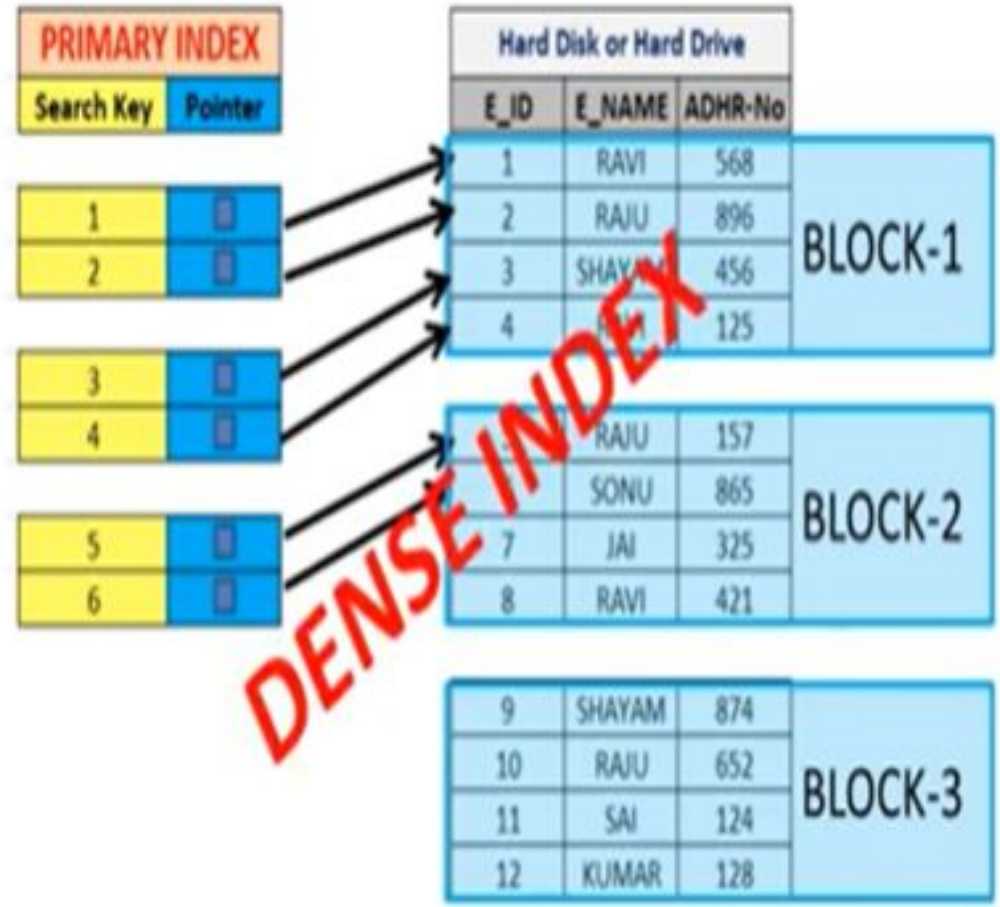
1. Unordered Data
2. Non-Key or Key based search required.
3. Data already have Primary Index

- Data searching requires MORE TIME as compared to the clustered index
- Secondary Index is of DENSE INDEX type, where ALL SEARCH KEY VALUE pointing to specific block.

INDEX – SECONDARY INDEX

PRIMARY INDEX	
Search Key	Pointer
1	
2	
3	
4	
5	
6	

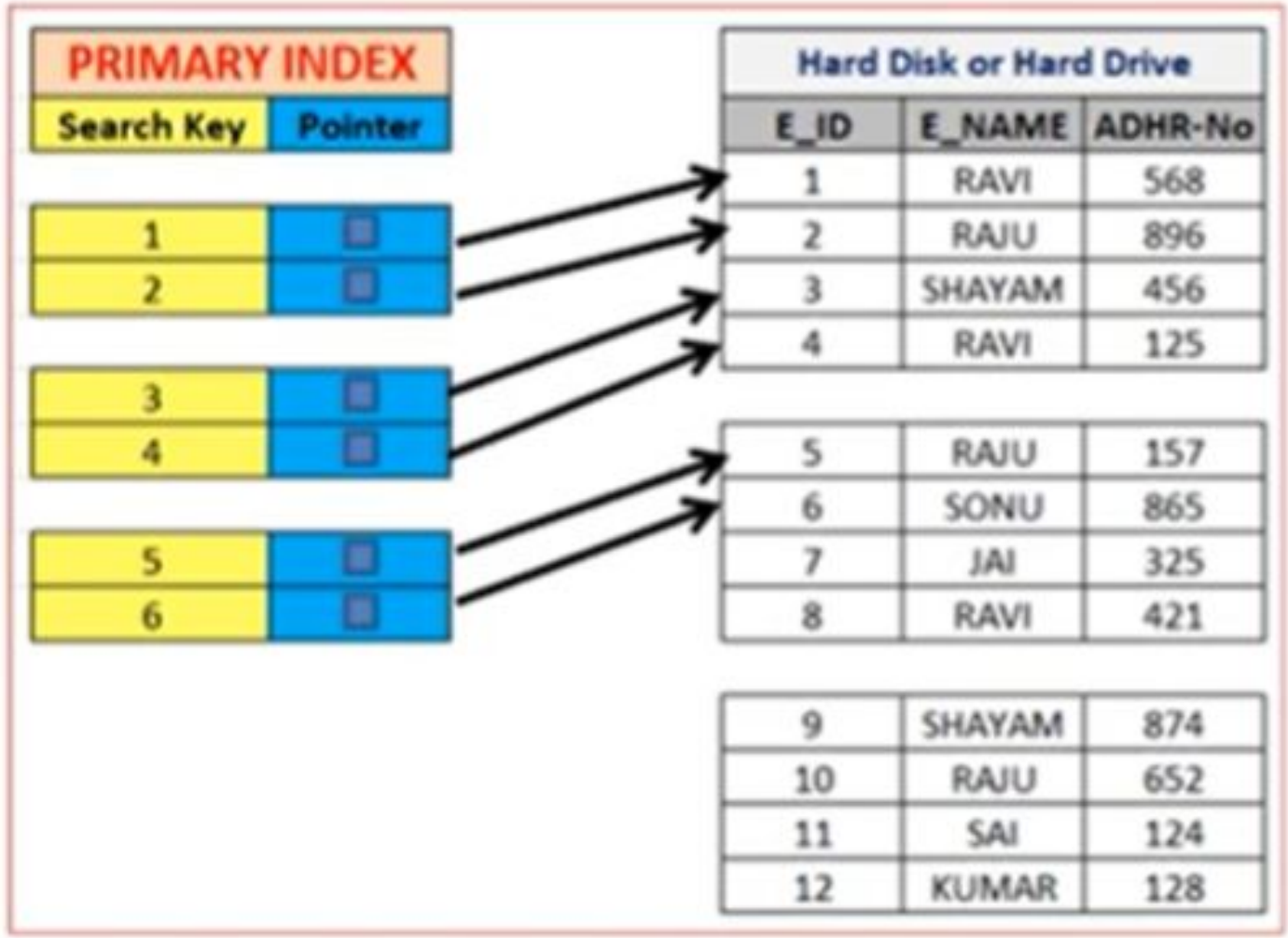
Hard Disk or Hard Drive		
E_ID	E_NAME	ADHR-No
1	RAVI	568
2	RAJU	896
3	SHAYAM	456
4	RAVI	125
5	RAJU	157
6	SONU	865
7	JAI	325
8	RAVI	421
9	SHAYAM	874
10	RAJU	652
11	SAI	124
12	KUMAR	128

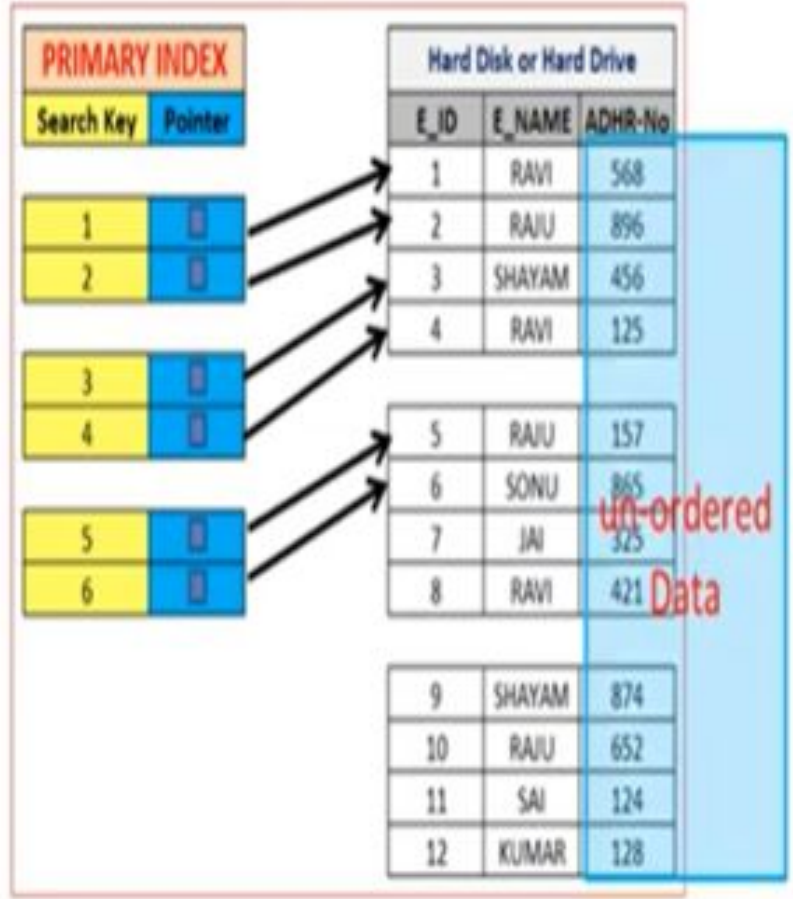


INDEX – SECONDARY INDEX

When you need to search data OTHER THAN KEY OF PRIMARY INDEX, such as Aadhar Card No, which is also a UNIQUE KEY IN DATABASE.

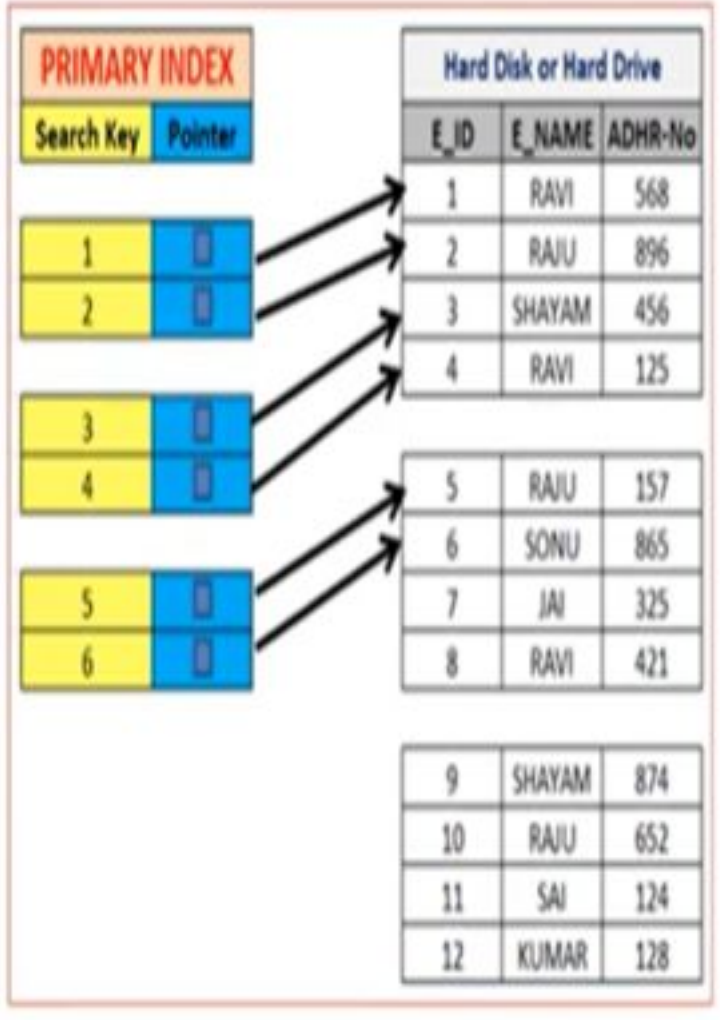
But not sorted



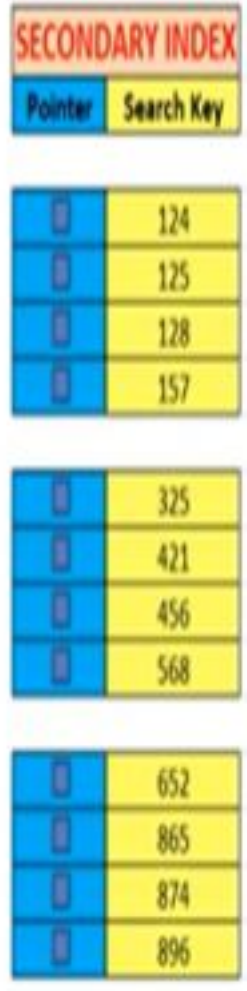


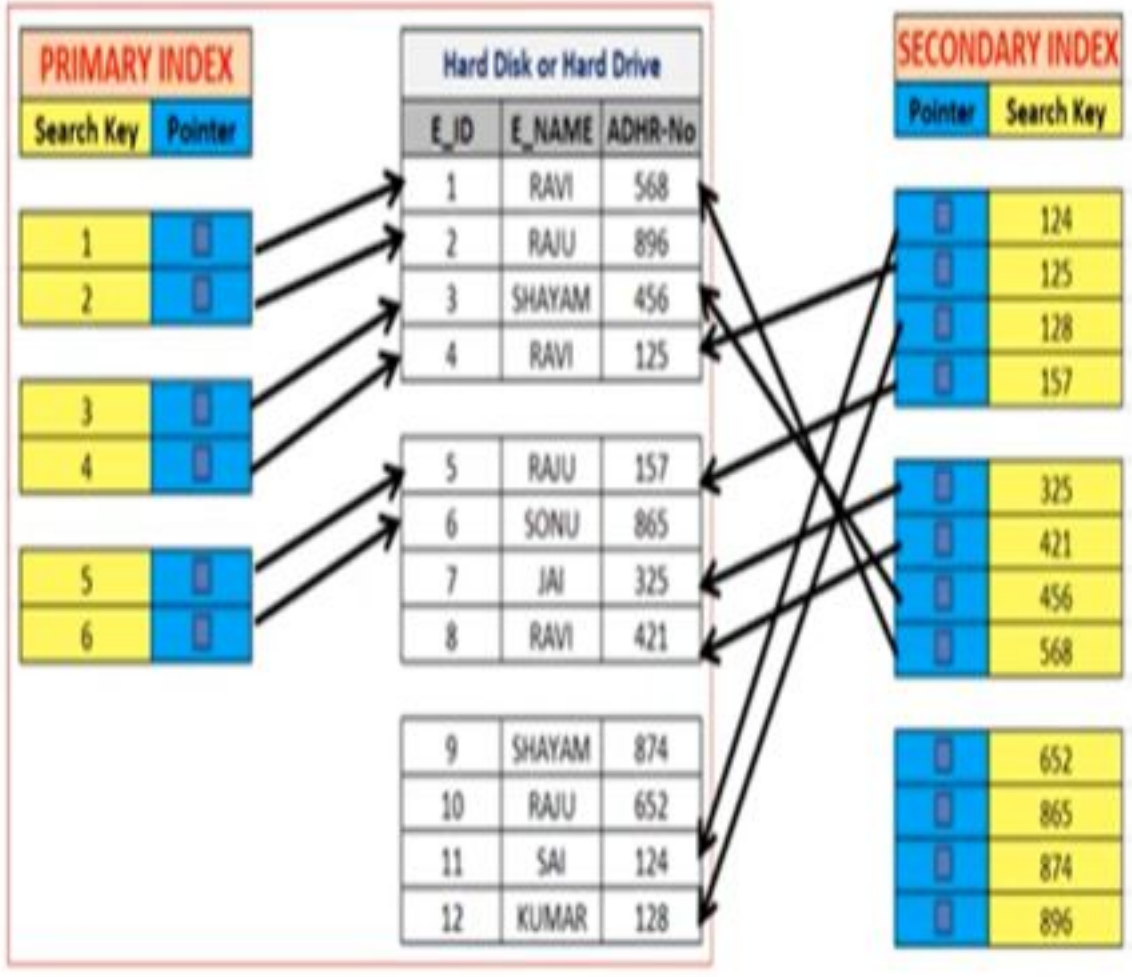
INDEX – SECONDARY INDEX

Step 1: Sort the Non-Key attribute

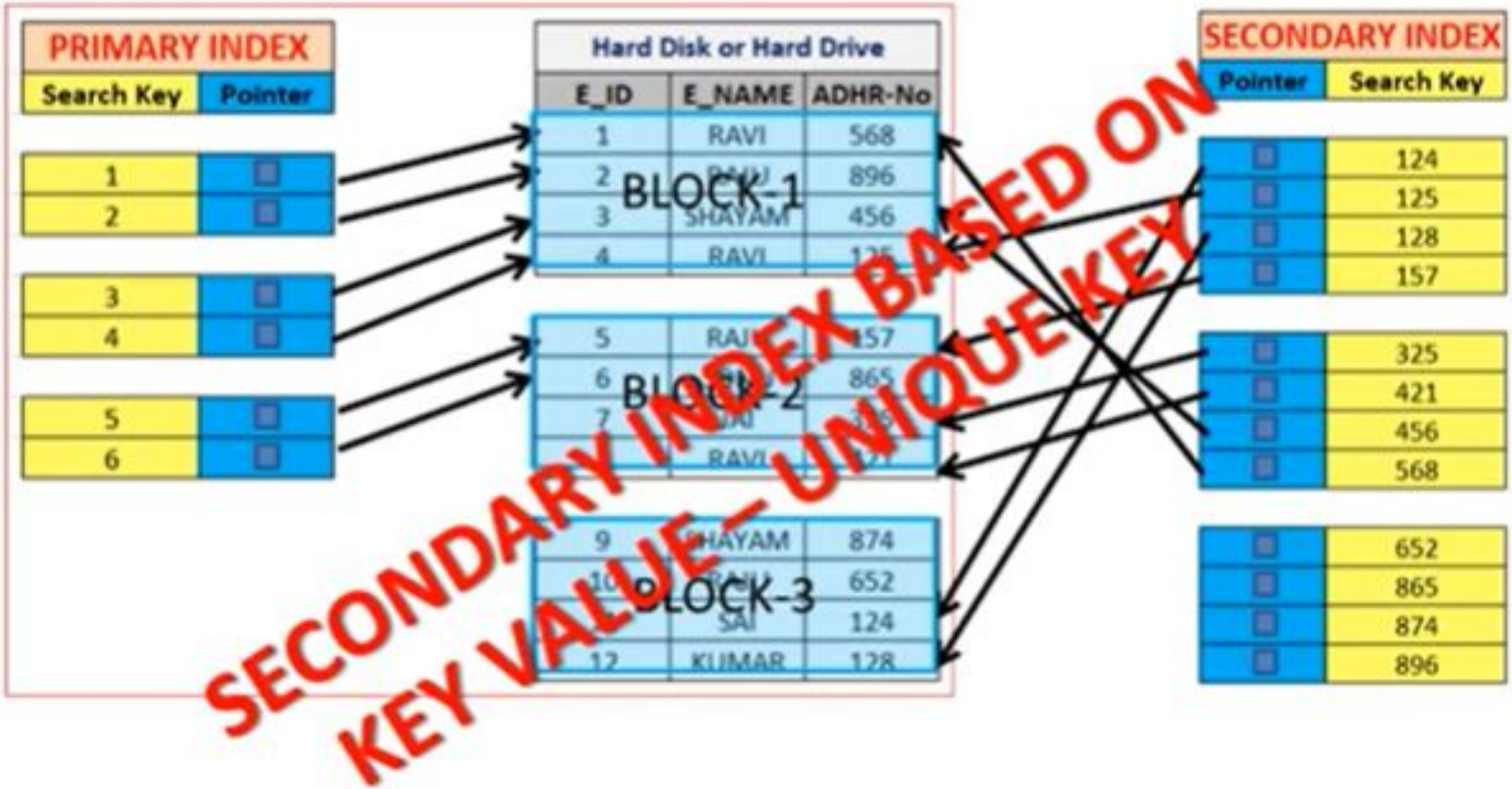


Step 2: Dense map Map every non-key attribute to the corresponding records



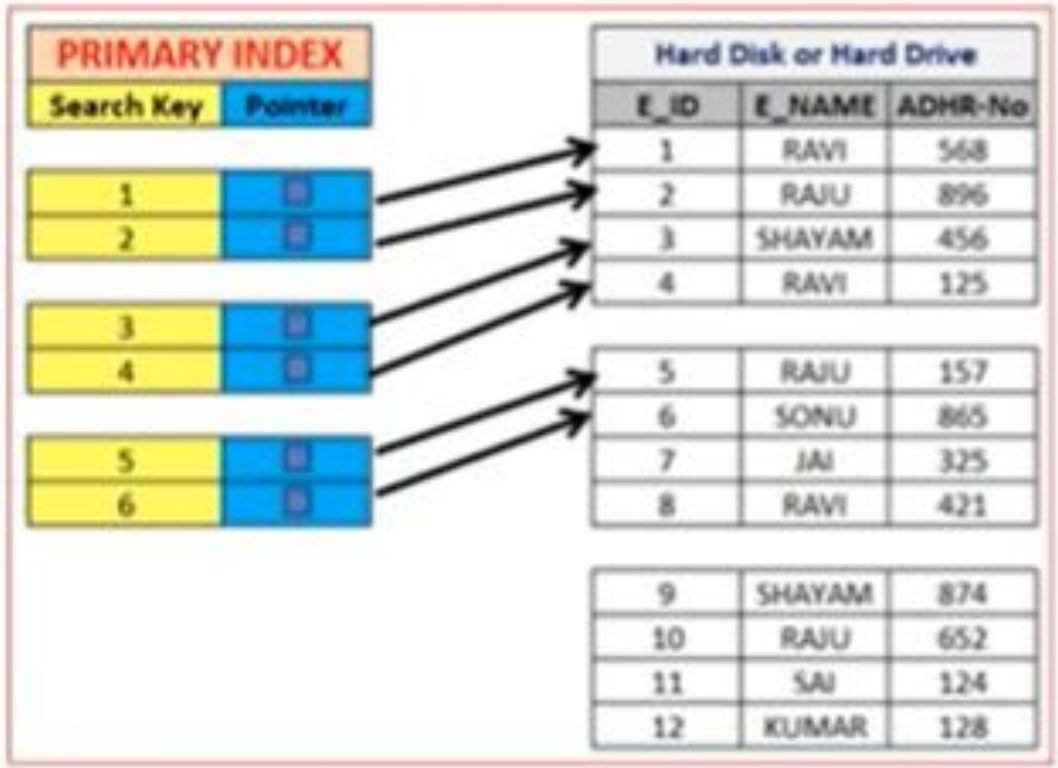


INDEX – SECONDARY INDEX

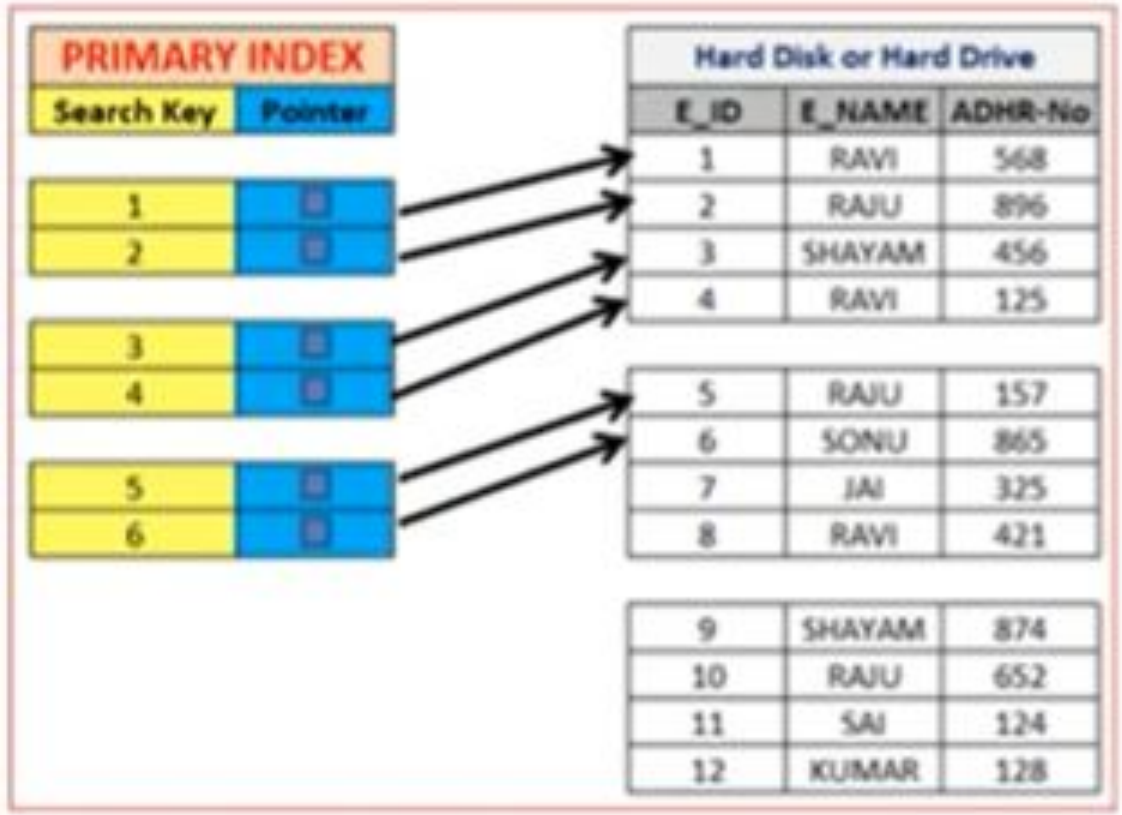


INDEX – SECONDARY INDEX

When you need to search data OTHER THAN KEY OF PRIMARY INDEX, such as E_Name, which is NON KEY and having duplicate values in table of a DATABASE.



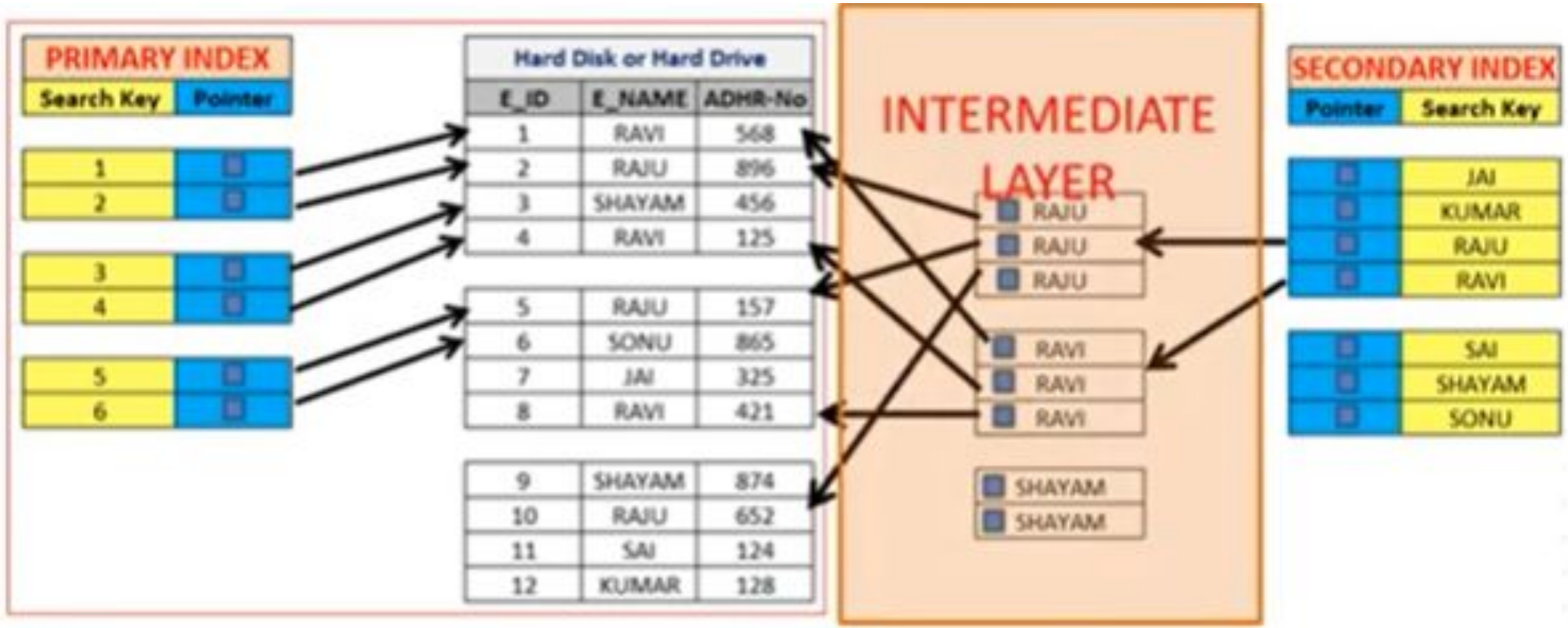
INDEX – SECONDARY INDEX







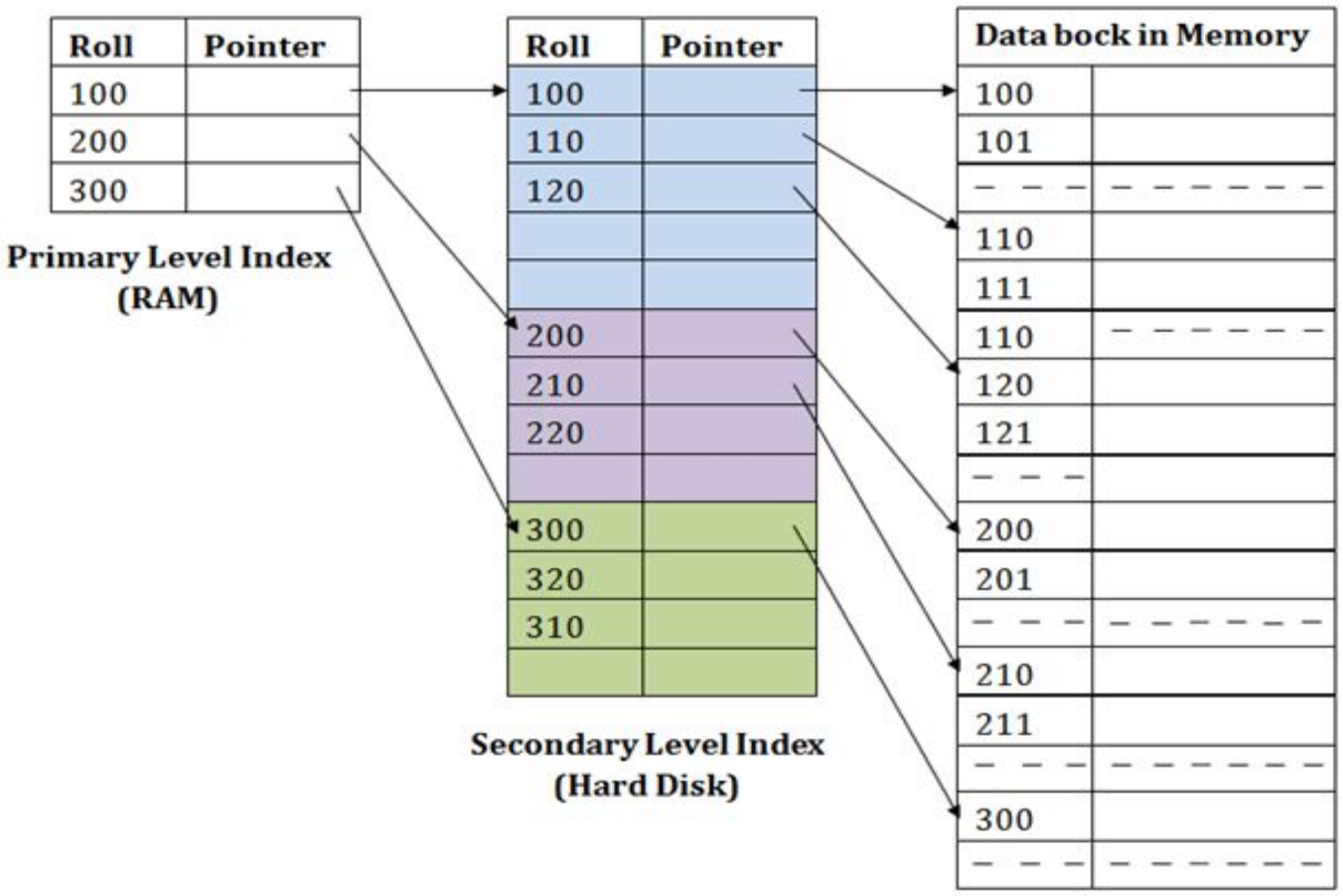
INDEX – SECONDARY INDEX



3. Secondary Index

- In the sparse indexing, as the size of the table grows, the size of mapping also grows. These mappings are usually kept in the primary memory so that address fetch should be faster. Then the secondary memory searches the actual data based on the address got from mapping.
- If the mapping size grows then fetching the address itself becomes slower. In this case, the sparse index will not be efficient. To overcome this problem, secondary indexing is introduced.
- In secondary indexing, to reduce the size of mapping, another level of indexing is introduced.
- In this method, the huge range for the columns is selected initially so that the mapping size of the first level becomes small. Then each range is further divided into smaller ranges. The mapping of the first level is stored in the primary memory, so that address fetch is faster. The mapping of the second level and actual data are stored in the secondary memory (hard disk).

3. Secondary Index



For example:

- If you want to find the record of roll 111 in the diagram, then it will search the highest entry which is smaller than or equal to 111 in the first level index. It will get 100 at this level

- Then in the second index level, again it does max (111) <= 111 and gets 110. Now using the address 110, it goes to the data block and starts searching each record till it gets 111

- This is how a search is performed in this method. Inserting, updating or deleting is also done in the same manner.

https://youtu.be/krrSzX7q30c?si=okYdty_3OW6e9C0w